

1 INTRODUCTION

The management of unstructured data, including video, image, and text, has become an urgent requirement [66]. A notable advancement in this domain has been the widespread adoption of learning-based embedding models, which leverage high-dimensional vector representations to enable effective and efficient analysis and search of unstructured data [37, 61]. High-dimensional Vector Similarity Search (HVSS) is a critical challenge in many domains, such as databases [25, 68], information retrieval [28, 32], recommendation systems [19, 54], scientific computing [51, 78], and large language models (LLMs) [7, 12, 44]. The computational complexity associated with exact query answering in HVSS has spurred recent research efforts toward developing approximate search methods [25, 49, 68]. While various compact index structures and intelligent search algorithms have been proposed to achieve a balance between efficiency and accuracy [42, 45, 68], the majority of these approaches assume that both the vector dataset and its corresponding index can be accommodated in the main memory (DRAM). However, in practical scenarios, the sheer volume of vector data often surpasses the capacity of the main memory, necessitating substantial expansion of main memory resources [4, 16, 40] to support state-of-the-art in-memory HVSS algorithms like NSG [25] and HNSW [49]. To illustrate, constructing an HNSW index based on one billion floating-point vectors in 96 dimensions would require more than 350GB of memory, resulting in out-of-memory failures [60] when executed on a typical single server [3]. Consequently, there is a growing demand for disk-based methods that leverage solid-state disks (SSDs) as a storage and retrieval medium for vector data, thereby circumventing the constraints imposed by main memory [16, 35, 74]. This paradigm shift allows for handling massive-scale vector datasets on a single machine, accommodating billions of vectors with improved efficiency and scalability.

However, maintaining a single large vector index on a solitary machine (Fig. 1(a)) proves impractical for vector databases, as it constrains many essential system features for industrial applications [29]. For example, constructing a sole DiskANN index for a dataset at a billion-scale may demand over five days and peak memory usage of 1,100GB [35], significantly impacting system availability. Additionally, a large index exhibits poor migration capabilities, essential for scaling, load balancing, and fault tolerance [29]. Consequently, **vector databases consistently partition large-scale data into multiple segments and assign an appropriate number of segments to a single machine** (Fig. 1(b)) [29, 41, 66]. Recent trends show that data fragmentation has become commonplace in mainstream vector databases [23, 29, 41]. Each segment operates with limited storage capacity and computing resources [4, 29], and an independent medium-sized index is constructed on each segment for autonomous searching. Leveraging specific data fragmentation strategies and a query coordinator allows us to search only *a few* segments of a machine during vector query execution [21, 29, 75]. This poses an open problem of efficient and accurate HVSS within a data segment in vector databases [6, 29]. In general, we may manage tens of millions of vectors on a typical segment with only several gigabytes of space [4, 29]. The space left for index storage is very limited. Therefore, HVSS within the segment necessitates a meticulous equilibrium between *search performance* (encompassing accuracy and efficiency) and *space cost*.

No existing work has achieved effective HVSS on the data segment, considering both search performance and space cost. An immediate attempt is to use existing disk-based methods on the segment to handle large-scale vector data. However, as reported in the literature [16, 60] and also empirically verified in our experiments (§6.2), these techniques are in a quandary in this scenario. For example, two state-of-the-art disk-based schemes—SPANN [16] and DiskANN [35]—lie in either of the two following extremes. At one extreme, SPANN [16] requires huge disk space to support efficient vector queries in the segment since each vector is copied multiple times (up to eight times [16]). Moreover, in vector databases, a segment may have multiple replicas that are distributed on

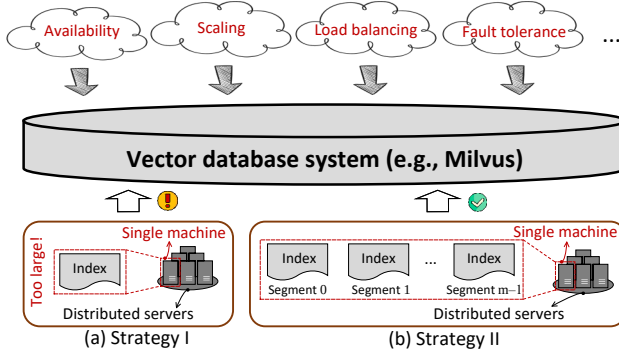


Fig. 1. Two indexing strategies on a single machine for vector database system. In a distributed setting, different machines share the same strategy [29].

different machines for fault tolerance [29]. This will further increase the disk space overhead for SPANN. At the other extreme, the vector index of DiskANN [35] can fit the segment's disk capacity but DiskANN suffers from high latency due to the large number of random disk I/Os during the search process [16, 35]. Therefore, it is particularly challenging to achieve a balance between search performance and space cost for HVSS on data segments.

We find that current disk-based methods for HVSS still follow the paradigm of memory-based solutions in their data layout and search strategy. This leads to numerous disk I/Os, which limit search performance under a given space cost. We illustrate this with DiskANN [35], the state-of-the-art disk-resident graph index algorithm¹ (cf. §2.2). First, each hop along the search path corresponds to a disk I/O request. The search procedure on the graph index executes a sequence of disk I/O requests, which results in high search latency. Second, the disk bandwidth is underutilized due to poor data locality. For each disk I/O in the search procedure, only one vertex's information (a vector and its neighbor IDs) is *relevant*, which typically takes a few hundred bytes. However, the smallest disk I/O unit is a block, which is usually 4KB in size. Therefore, most of the data read from the disk is wasted. According to our evaluation on the BIGANN dataset [3], DiskANN's disk I/O operation takes up to 92.5% of the time in the search procedure, and up to 94% of the vertices (they are *irrelevant*) in each loaded block are wasted (i.e., the vertex utilization ratio is low). Thus, there is much potential to achieve an I/O-efficient HVSS using graph index on the data segment.

In this paper, we introduce an I/O-efficient disk-resident graph index framework tailored for high-dimensional vector similarity search on a data segment, called Starling. Starling optimizes data layout and search strategy to improve the search performance without additional space cost. The data layout consists of a sampled in-memory navigation graph and a reordered disk-based graph, which help find query-aware dynamic entry points and improve data locality, respectively. Based on this data layout, we design a search strategy that reduces disk I/Os by shortening the search path and increasing the vertex utilization ratio. Specifically, Starling builds a navigation graph in memory by sampling a small portion of vectors. This allows the search procedure to quickly find some vertices that are close to the query vector without disk I/O, and then start searching on the disk-based graph from these vertices. To improve data locality on disk, Starling reorganizes the graph index layout by shuffling the vertices and storing them with their neighbors in the same block (i.e., *block shuffling*). This way, one disk I/O can load information from multiple relevant vertices in the search path. Based on this, we design a block search strategy that checks all the data from disk I/O and adds the ones that are potentially useful to the search sequence. We

¹We exclude SPANN as it duplicates each vector up to eight times, which far exceeds the capacity of the data segment when serving tens of millions of vectors.

further improve the strategy by three computation-specific optimizations. Finally, we also propose efficient algorithms for approximate nearest neighbor search (ANNS) and range search (RS) queries based on the optimized search procedure.

To the best of our knowledge, this is the first work that tackles the fundamental data locality problem of disk-based graph index for the data segment by effective block shuffling algorithms. Our evaluation shows that Starling achieves 43.9× higher throughput with 98% lower query latency than state-of-the-art methods under the same accuracy. The main contributions of this work are:

- We present Starling, an I/O-efficient disk-resident graph index framework with excellent universality that enhances the search efficiency of various graph index algorithms (e.g., Vamana [35], NSG [25], and HNSW [49]) on a data segment (§3, §6.7).
- We design a data layout for the data segment that consists of two components: an in-memory navigation graph (§4.2) and a reordered disk-based graph (§4.1). This data layout reduces the search path length and increases the vertex utilization ratio of each loaded block when searching on the disk-based graph index.
- We prove that the block shuffling problem is NP-hard and has no polynomial time approximation algorithm with a finite approximation factor unless $P=NP$ (§4.1). Hence, we devise three effective heuristic shuffling algorithms that improve the locality of the disk-based graph index (§4.1).
- We propose a block search strategy that exploits the data locality of the reorganized index layout to reduce disk I/Os (§5.1). Based on this strategy, we also provide three computation-specific optimizations (§5.1) and develop search algorithms for two major types of queries in vector databases (§5.2 and §5.3).
- We implement Starling and evaluate it on four real-world datasets to verify its effectiveness, efficiency, and scalability (§6). We show that search efficiency can be improved significantly by simply adjusting the index layout on the disk.

2 PRELIMINARIES

In this section, we formulate two critical queries in vector databases. Then we explain why current methods are inefficient on the data segment and state our optimization goal in this paper.

2.1 Two Query Types for HVSS

K Nearest Neighbor Search (KNNS). Given a vector dataset $X = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^D$, a query vector $q \in \mathbb{R}^D$, a distance function $dist(\mathbb{R}^D, \mathbb{R}^D) \rightarrow \mathbb{R}$, and a positive integer k ($0 < k < n$), the KNNS problem finds a set R_{knn} of k vectors from X that are closest to q such that for any $x_i \in R_{knn}$ and $x_j \in X \setminus R_{knn}$,

$$dist(x_i, q) \leq dist(x_j, q) \quad . \quad (1)$$

The distance function can be Euclidean distance or others.

The KNNS problem requires scanning all the vectors in the dataset, which is impractical for large vector datasets. Hence, most studies try to find approximate results that are close to the exact KNNS results. This is the Approximate Nearest Neighbor Search (ANNS) problem [1, 24, 25, 49]. ANNS uses $Recall \in [0, 1]$ to measure the accuracy of approximate results. Let R'_{knn} be the approximate results, then $Recall$ is

$$Recall = \frac{|R_{knn} \cap R'_{knn}|}{k} \quad . \quad (2)$$

A higher recall means a more accurate result. Many studies have designed different index structures for vector data to improve the trade-off between search efficiency and accuracy.